

Special Participation C: Refactoring HW 11 Q3 (Transformer Interpretability)

Athul Krishnan, Mishty Dhekial, Jaimyn Drake

1 Overview

For Special Participation C, our group decided to refactor the provided code for Homework 11 Question 3: Transformer Interpretability. We wanted to reformat the single notebook into an easier-to-follow modular python package, where tests, utils, and core logic are all clearly separated in the file tree. Additionally, we wanted to adhere to PEP8 style [2], PEP257 docstring [1], and PEP484 type hinting [3] guidelines, along with commonly used style practices such as the Numpy Style Guide [5], and Google's Python Style guide [4]. With this, we wanted to ensure that the code is more readable, maintains its pedagogical value, and is presented in a format that more closely follows typical software engineering practice seen in research work/industry (as per the Participation C prompt). We have included refactored versions of both a blank template for students, and a verified solution.

The full refactored codebase can be found on GitHub at:

<https://github.com/a-thul/CS.C182-Special-Participation-C-HW11-Q3/tree/main>.

2 Updated File Tree

The new structure is as follows:

```
CS_C182-Special-Participation-C-HW11-Q3/
├── transformer_interpretability/ ..... Reference implementation
│   ├── __init__.py
│   ├── core/
│   │   ├── __init__.py
│   │   ├── attention.py ..... single_attention_head implementation
│   │   └── induction_heads.py ..... induction_copy_head + helpers
│   ├── utils/
│   │   ├── __init__.py
│   │   ├── numerical.py ..... softmax, create_causal_mask
│   │   └── constants.py ..... EPSILON, MASK_VALUE, VOCAB_SIZE, etc.
│   ├── tests/
│   │   ├── __init__.py
│   │   ├── test_attention.py
│   │   ├── test_induction_heads.py
│   │   ├── test_numerical.py
│   │   └── test_cases/
│   │       ├── single_attention_head_test_cases.json
│   │       └── induction_head_test_cases.json
│   └── q_code_interpretability_refactored.ipynb
└── README.md
```

3 Detailed Refactoring Changes

3.1 From Notebooks to Packages

Originally, `q_code_interpretability.ipynb` and `q_code_interpretability_sol.ipynb` interleaved:

- problem text, hints, and TODOs in markdown,
- function definitions for `single_attention_head` and `induction_copy_head`,
- global constants defined in cells, and
- inline test code that opened JSON files and asserted on outputs.

In our refactor:

- the core logic (where students will implement TODOs) moved into `core/attention.py` and `core/induction_heads.py`;
- numerical utilities and constants moved into `utils/numerical.py` and `utils/constants.py`;
- all tests and fixtures now live under `tests/`, run with `pytest`; and
- the refactored notebook `q_code_interpretability_refactored.ipynb` focuses on imports, sanity checks, and explanations rather than defining the functions itself. However, the notebook

directs students the specific files that they need to implement, which contain detailed docstrings conceptually (without giving away hints for the code) explaining the importance of each step they need to implement.

The student template package follows the same structure but leaves the key functions as TODOs, preserving the original “fill in the blanks” experience while exposing students to a realistic project layout.

3.2 Student Template vs. Original Notebook

We refactored the original student-facing notebook `q_code_interpretability.ipynb` into a template package `transformer_interpretability_template`.

Concretely, the template package mirrors the original notebook in the following ways:

- The two functions students are asked to implement in the notebook, `single_attention_head` and `induction_copy_head`, are now defined in `transformer_interpretability_template/core/attention.py` and `transformer_interpretability_template/core/induction_heads.py` with the *same* signatures and high-level step breakdown as in the original notebook (e.g., “Step 0: convert to numpy arrays”, “Step 2: apply causal mask”, etc.).
- Instead of minimal inline docstrings in the notebook, the template modules have NumPy-style docstrings and module-level documentation that describe shapes, types, and the interpretability context (including references to Vaswani et al. and Olsson et al.). The TODO blocks are still present, but they sit underneath step-by-step specifications similar to the original notebook.
- The original notebook declared global constants like `seed`, `eps`, and `vocab_size` in cells. In the template, all such “magic numbers” are centralized in `transformer_interpretability_template/utils/constants.py` (e.g., `EPSILON`, `VOCAB_SIZE`, `D_MODEL`), following the principle of avoiding scattered literals.
- Numerical helpers that were implicit in the notebook (softmax, causal mask construction) are provided to students in `utils/numerical.py`. The template core functions import `softmax` and `create_causal_mask` rather than re-implementing them inline, which both reduces boilerplate and nudges students toward reusing utilities.
- The original notebook loaded JSON test cases directly in cells. In the refactor, those same JSON files live under `transformer_interpretability_template/tests/test_cases/`, and the test logic is encapsulated in proper `pytest` test modules. The behavior students must satisfy is unchanged; only the location and organization of the tests have improved.
- Finally, the new `transformer_interpretability_template/q_code_interpretability_refactored.ipynb` replaces the old “!git clone + %cd” workflow with standard imports from the template package. From a student’s point of view, they still open a notebook, fill in TODOs, and run tests; under the hood, they are now editing a small, well-structured library.

Overall, `transformer_interpretability_template` is a direct translation of the original student notebook into package form: the pedagogical structure (TODOs, hints, and tests) is preserved, but wrapped in better engineering practices (modules, centralized constants, shared numerical utilities, and a test directory).

3.3 Core Logic: Attention and Induction Heads

In the original solution notebook, the attention head:

- manually converts to NumPy arrays,
- builds a causal mask with `np.triu`,
- uses a hard-coded `-1e9` for masked entries, and
- implements softmax inline using `np.exp` and division.

In the refactored `core/attention.py`, we:

- add PEP 484 type hints (using a `MatrixInput` alias and `NDArray[np.floating]`),
- write NumPy-style docstrings that clearly describe shapes and return values,
- reuse `softmax` and `create_causal_mask` from `utils/numerical.py`, and
- centralize `MASK_VALUE` in `utils/constants.py` instead of hard-coding it.

The induction head in the original notebook was a single monolithic function that built weights, ran two attention heads, added a residual connection, and returned logits. In `core/induction_heads.py`, we split this into small helpers (for building previous-token and copy-head matrices) plus a well-documented public `induction_copy_head` function, making the underlying flow (previous-token head → residual connection → copying head) much clearer [6].

3.4 Utilities, Constants, and Tests

The original notebooks declared global constants like `seed`, `eps`, and various dimensionalities inside cells. We centralized all such “magic numbers” in:

- `transformer_interpretability/utils/constants.py` (and the mirrored file in the template), which defines `EPSILON`, `MASK_VALUE`, `VOCAB_SIZE`, `D_MODEL`, and token-to-ID mappings.
- `utils/numerical.py`, which implements a numerically stable `softmax` and `create_causal_mask(seq_len, mask_value=-1e9)` with input validation.

Instead of inline assertions and ad hoc JSON loading, we now have:

- `tests/test_attention.py` and `tests/test_induction_heads.py`, which load `single_attention_head_test_cases.json` and `induction_head_test_cases.json` (unchanged from existing tests) and verify if outputs are within within margin `EPSILON`;
- `tests/test_numerical.py`, which checks softmax and causal mask properties
- thin refactored notebooks that import from the packages and run all tests that were in the previously existing notebook.

4 Style Guidelines and Citations

Our refactoring was guided by established style and engineering references:

- **PEP 8** [2] for naming, layout, and code organization (e.g., module-level docstrings, import ordering, line length).
- **PEP 257** [1] for docstring conventions.
- **PEP 484** [3] for type hints (e.g., use of `typing.Union`, `NDArray` from `numpy.typing`).
- **Google Python Style Guide** [4] for testing conventions, module docstrings, and avoidance of magic numbers.
- **NumPy / numpdoc style** [5] for structured docstrings with `Parameters`, `Returns`, and `Notes` sections.
- **Mechanistic interpretability references:** Vaswani et al. (transformer attention) and Olsson et al. (induction heads) to align comments and docstrings with the literature [6].

5 LLM Usage

Per the given Participation C prompt, we incorporated LLM assistance:

- We used Claude Code to initially refactor the assignment into a more modular package, with an initial pass using outlined style conventions via the following prompt:
 - “Hi Claude! I’m working on refactoring an existing notebook ‘q_code_interpretability.ipynb’ into a modular scheme that is easier to parse (e.g. splitting into tests/utils/main). I would like to reformat the notebook into a scheme that follows a modular format/style guidelines, more consistent with industry practices. I would like you to entirely adhere to the following guidelines throughout the entire refactoring process:
PEP8: <https://peps.python.org/pep-0008/> - overall style
PEP484: <https://peps.python.org/pep-0484/> - type hinting
PEP257: <https://peps.python.org/pep-0257/> - docstring conventions
Feel free to also refer to Google’s style guide, as an example of an industry standard style guide.
Google’s Python Style Guide: <https://google.github.io/styleguide/pyguide.html>
The entire notebook serves as a homework question for a deep learning course; the refactored code should preserve/enhance pedagogical value, encouraging students to learn the intuition behind transformers’ inner workings while not giving too much away about the code solutions.
I have also attached the existing solution notebook (q_code_interpretability_sol.ipynb) with the staff solution, for your reference.”

We found that the originally generated comments/docstrings often gave away too much information on the solution in the student template, compromising learning, so we modified them to give more of a conceptual overview/rundown, as well as motivations on why each step matters. With this, we hope that students can more logically derive the code and understand its intuitive function, rather than pattern matching or copying down formulas.

All reimplemented code has been validated on the provided unit tests from the original homework; the LLM was used primarily for style, verbose/helpful docstrings and comments, creating a README, and code reorganization.

References

- [1] David Goodger, Guido van Rossum. *PEP 257 – Type Hints*. <https://peps.python.org/pep-0257/>.
- [2] Guido van Rossum, Barry Warsaw, and Nick Coghlan. *PEP 8 – Style Guide for Python Code*. <https://peps.python.org/pep-0008/>.
- [3] Guido van Rossum et al. *PEP 484 – Type Hints*. <https://peps.python.org/pep-0484/>.
- [4] Google. *Google Python Style Guide*. <https://google.github.io/styleguide/pyguide.html>.
- [5] Numpy developers. *NumPy Documentation Style Guide*. <https://numpydoc.readthedocs.io/en/latest/format.html>.
- [6] Catherine Olsson et al. *In-context Learning and Induction Heads*. arXiv preprint arXiv:2209.11895, 2022.